

Programação Orientada a Objetos

Herança e Polimorfismo no GestorOS

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Herança: a Superclasse Concentra o Comum

Cliente e o futuro Técnico compartilham id, nome e telefone, cada um validado exatamente da mesma forma nas duas classes. Herança extrai esse comum para uma única classe, Pessoa, e cada subclasse passa a herdar os atributos e métodos sem reescrevê-los: o que muda de uma classe para a outra é só o que de fato é exclusivo de cada uma.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Por Que Pessoa É Abstrata

No domínio do GestorOS não existe "uma Pessoa" solta: só existem Clientes e Técnicos, cada um com um papel definido na assistência técnica. Declarar Pessoa abstract impede a instrução `new Pessoa(...)` em qualquer lugar do código, e obriga toda instância concreta a nascer de uma subclasse que diga, de fato, o que ela é.



Prof. Everaldo Graciliano Souza Ribeiro

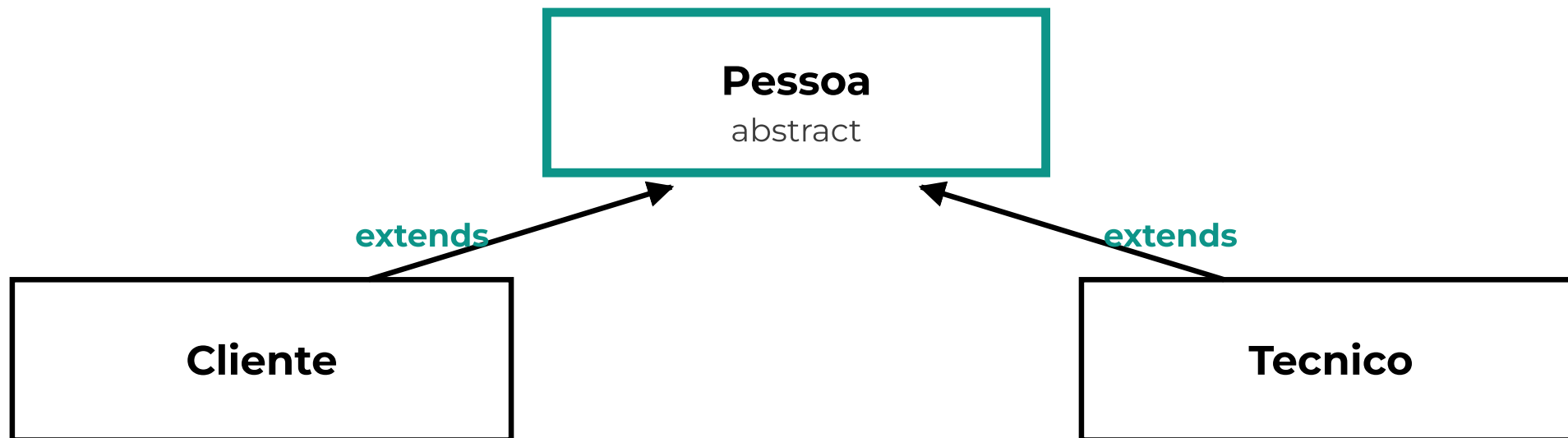
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Cliente e Tecnico Estendem Pessoa



Cliente e Tecnico herdam id, nome, telefone e a validação; nenhuma das duas repete esse código.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

super(): Delegando ao Construtor da Superclasse

Cliente e Tecnico não repetem a validação de nome e telefone: seus construtores chamam `super(nome, telefone)`, e é o construtor de Pessoa quem executa `setNome` e `setTelefone` pela primeira vez. A mesma regra de validação passa a valer, sem duplicação de código, para qualquer subclasse que Pessoa venha a ganhar no futuro.



Pessoa: Construtor e Método Abstrato

```
protected Pessoa(String nome, String telefone) {  
    // Delega a validacao para os proprios setters, chamados uma vez.  
    setNome(nome);  
    setTelefone(telefone);  
}  
  
public final void setNome(String nome) {  
    if (nome == null || nome.trim().length() < 3) {  
        throw new IllegalArgumentException("Nome deve ter ao menos 3 caracteres.");  
    }  
    this.nome = nome.trim();  
}  
  
public abstract String getPapel();
```



Método Abstrato: Contrato Sem Implementação

`getPapel()`, declarado abstrato em `Pessoa`, não tem corpo: nenhuma chave, nenhuma instrução, só a assinatura do método. Cada subclasse concreta é obrigada, pelo próprio compilador, a fornecer sua implementação — `Cliente` devolve "`Cliente`", `Tecnico` devolve "`Tecnico`" — e uma subclasse que esquecesse de implementar `getPapel()` simplesmente não compilaria.



Cliente.java: o Que Sobrou Depois da Herança

```
public class Cliente extends Pessoa {  
  
    public Cliente(String nome, String telefone) {  
        super(nome, telefone);  
    }  
  
    @Override  
    public String getPapel() {  
        return "Cliente";  
    }  
}
```



Tecnico: o Que É Exclusivamente Seu

Tecnico herda id, nome, telefone e as validações de Pessoa sem reescrever uma linha sequer delas; acrescenta apenas o que lhe pertence de fato — a especialidade, com sua própria validação e seu próprio getter. É o mesmo princípio de encapsulamento das aulas anteriores, agora combinado com herança: cada classe cuida só do que é seu.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Tecnico.java: a Nova Subclasse

```
public class Tecnico extends Pessoa {  
  
    private String especialidade;  
  
    public Tecnico(String nome, String telefone,  
                   String especialidade) {  
        super(nome, telefone);  
        setEspecialidade(especialidade);  
    }  
  
    @Override  
    public String getPapel() {  
        return "Tecnico";  
    }  
}
```



Sobrescrita e super(): Estender Sem Duplicar

O `toString()` de `Tecnico` chama `super.toString()`, que já monta "Tecnico: nome - telefone" a partir do método herdado de `Pessoa`, e apenas acrescenta a especialidade ao final. Sobrescrever um método não significa reescrevê-lo do zero: o comportamento da superclasse continua disponível através de `super`, e a subclasse só adiciona o que muda.



Prof. Everaldo Graciliano Souza Ribeiro

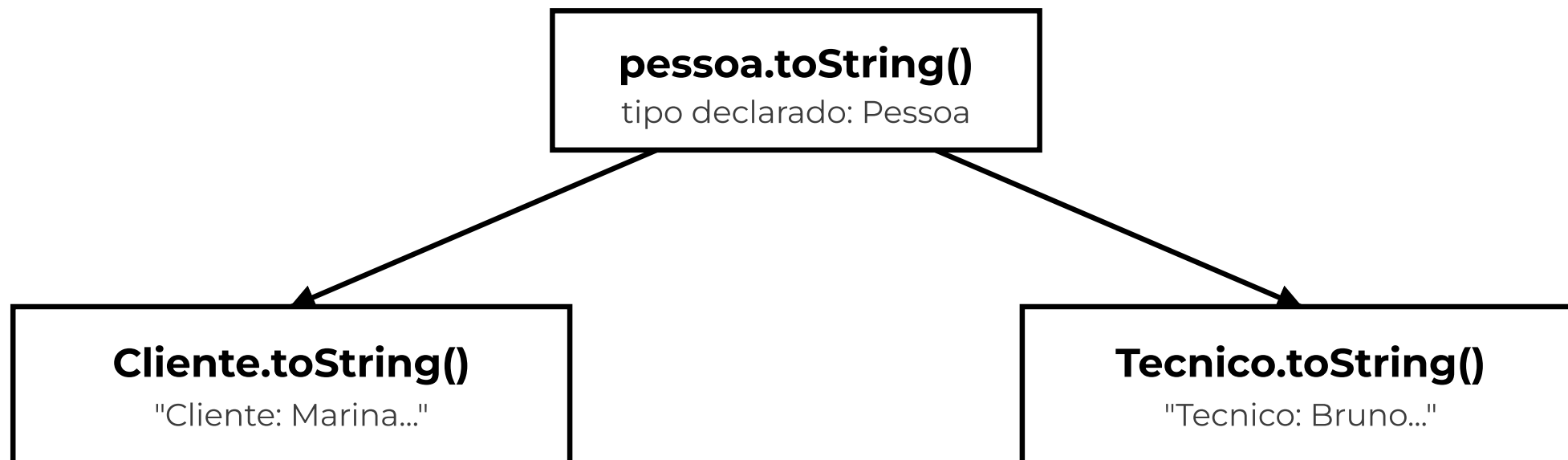
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Mesma Chamada, Dois Métodos Possíveis



O método que roda depende do **objeto real** guardado na variável, não do tipo Pessoa declarado nela.



Polimorfismo: Mesma Chamada, Comportamento Diferente

No laço for (`Pessoa pessoa : pessoas`), a variável `pessoa` é declarada `Pessoa`, mas o método que roda em `pessoa.toString()` depende do objeto real guardado ali a cada iteração — decidido apenas em tempo de execução, nunca pelo tipo declarado da variável. É esse mecanismo, e não uma cadeia de `if/instanceof`, que separa o comportamento de um `Cliente` do de um `Tecnico`.



GestorOS: Polimorfismo em uma Lista

```
List<Pessoa> pessoas = new ArrayList<>();
pessoas.add(cliente);
pessoas.add(tecnico);

// pessoa e Pessoa, mas o toString() que roda depende do objeto real.
for (Pessoa pessoa : pessoas) {
    System.out.println(" " + pessoa);
}
```



Atividades

Exercício 1

Por que Pessoa é declarada abstract, e o que o compilador impediria se ela não fosse?

Exercício 2

Explique, com as próprias palavras, o que super(nome, telefone) faz no construtor de Cliente e de Tecnico.

Exercício 3

No laço for (Pessoa pessoa : pessoas), o que decide qual toString() roda em cada iteração: o tipo declarado da variável ou o objeto guardado nela?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Criar um método abstrato `calcularComissao(BigDecimal valorOS)` em `Pessoa`, implementado só de forma significativa em `Tecnico` (Cliente pode devolver `BigDecimal.ZERO`).
2. Sobrescrever `equals()` e `hashCode()` em `Pessoa` a partir do `id`, e discutir por que as subclasses não precisam reimplementá-los.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.

